

BrandOrbit: AI-Driven Multi-Brand Social Media Content Management System

Detailed Design Document (DDD)



Supervisor

Nomaan Khan

Submitted by

Shahid Yasin

BSCS-E1-22-03

**Department of Computer Science,
University of Sahiwal.**

20/05/2026

1. System Architecture & Tech Stack

1.1 Executive Summary

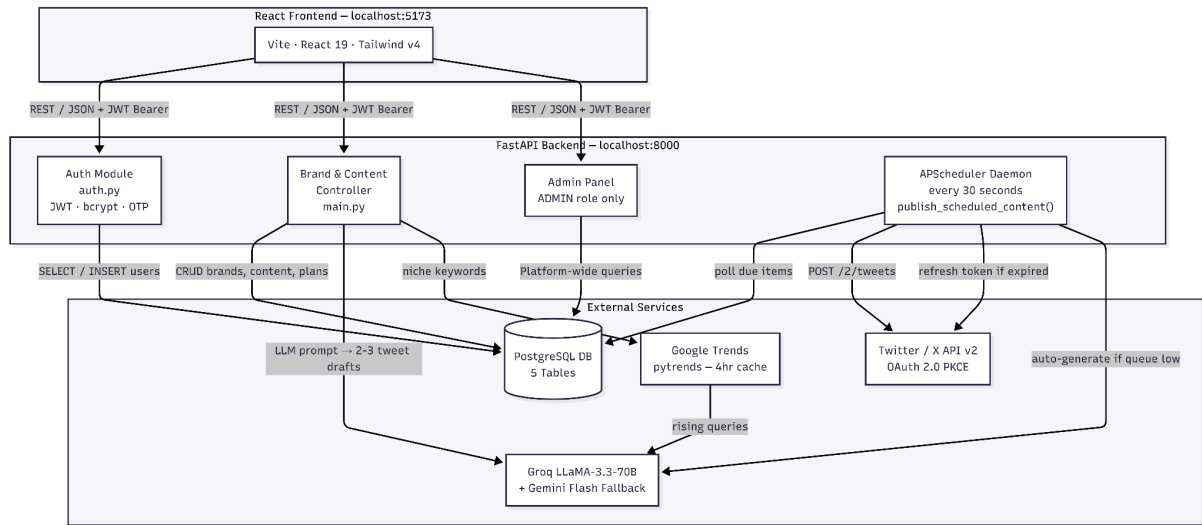
BrandOrbit is an AI-driven, multi-brand social media content management and automation platform. Its backend is a Python FastAPI monolith that serves as the single source of truth for stateless JWT authentication, multi-tenancy brand management, AI content generation via Groq LLaMA-3.3-70B (with Google Gemini fallback), and automated publishing via an APScheduler background daemon.

1.2 Technology Stack

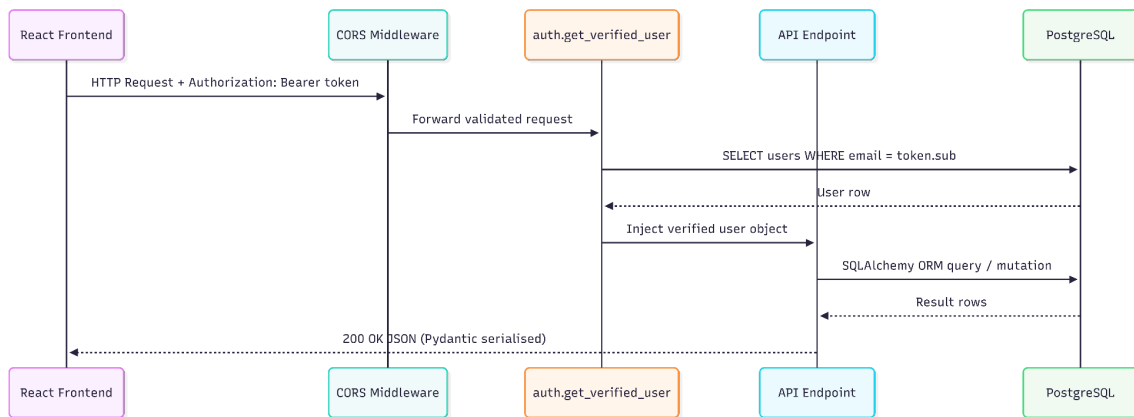
Layer	Technology	Purpose
API Framework	FastAPI 0.136	HTTP request handling, dependency injection, OpenAPI docs
Language	Python 3.14	Runtime environment
Database	PostgreSQL (via pgAdmin)	Persistent relational data storage
ORM	SQLAlchemy 2.0	Database model mapping and query building
DB Driver	pg8000 1.31	Pure-Python PostgreSQL adapter
Schema Validation	Pydantic v2	Request/response validation and serialisation
Auth — JWT	python-jose (HS256)	Stateless Bearer token issuance and verification
Auth — Passwords	passlib + bcrypt	Salted password hashing

AI — Primary	Groq LLaMA-3.3-70B	High-speed LLM inference for content generation and trend refinement
AI — Fallback	Google Gemini Flash	Secondary LLM if Groq API is unavailable
Trend Data	pytrends	Real-time Google Trends scraping
Twitter/X	Twitter API v2 + OAuth 2.0 PKCE	Authenticated social media publishing
Background Jobs	APScheduler	Interval-based scheduled post publishing every 30s

1.3 High-Level Architecture

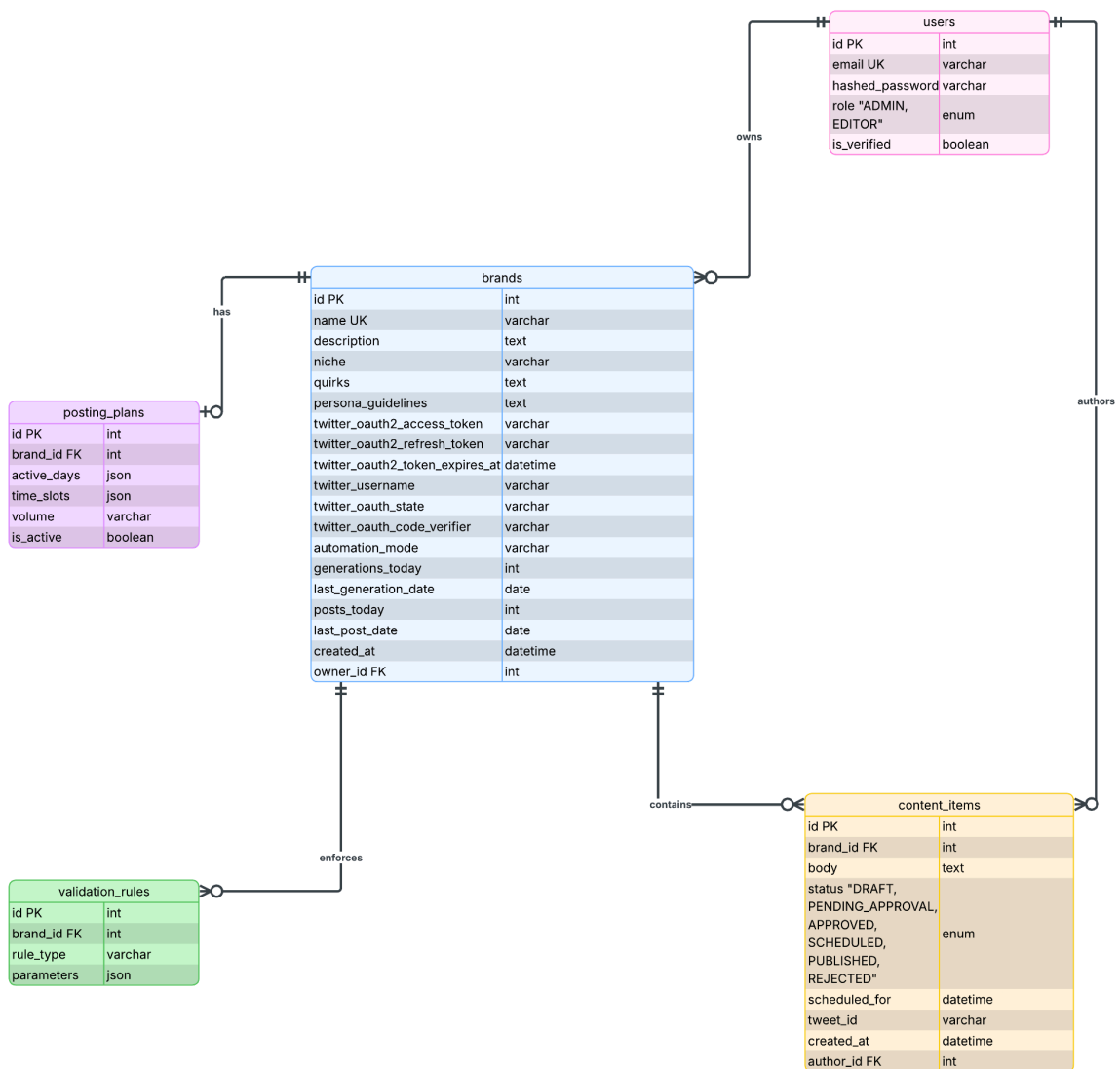


1.4 Data Flow



2. Database Design

2.1 Entity-Relationship Model



Cascade Rules:

- Deleting a **User** cascades → deletes all owned **Brand** records.
- Deleting a **Brand** cascades → deletes all **ContentItem**, **ValidationRule**, and **PostingPlan** records.

2.2 Data Dictionary

Table: **users**

Column	Type	Constraints	Description
id	INTEGER	PK, auto-increment	Unique system identifier
email	VARCHAR	UNIQUE, NOT NULL	Login credential and unique identity
hashed_password	VARCHAR	NOT NULL	bcrypt-hashed password
role	ENUM	DEFAULT EDITOR	Access level (ADMIN or EDITOR)
is_verified	BOOLEAN	DEFAULT FALSE	Set to TRUE after successful OTP verification

Table: **brands**

Column	Type	Constraints	Description
id	INTEGER	PK, auto-increment	Unique brand identifier

name	VARCHAR	UNIQUE, NOT NULL	Display name of the brand
description	TEXT	NULLABLE	Short human-readable brand description
niche	VARCHAR	NULLABLE	Topic category (e.g., "technology"). Required for AI
quirks	TEXT	NULLABLE	Brand personality traits for the LLM
persona_guidelines	TEXT	NULLABLE	Explicit content rules and tone guidance
twitter_oauth2_access_token	VARCHAR	NULLABLE	Short-lived OAuth 2.0 Bearer token
twitter_oauth2_refresh_token	VARCHAR	NULLABLE	Long-lived Refresh Token
twitter_oauth2_token_expires_at	DATETIME	NULLABLE	UTC expiry timestamp of the access token

twitter_username	VARCHAR	NULLABLE	Connected X/Twitter handle
twitter_oauth_state	VARCHAR	NULLABLE	Transient CSRF protection nonce for PKCE
twitter_oauth_code_verifier	VARCHAR	NULLABLE	Transient PKCE code verifier
automation_mode	VARCHAR	DEFAULT manual	manual, semi-automated, or auto
generations_today	INTEGER	DEFAULT 0	Daily generation counter (max 4)
last_generation_date	DATE	NULLABLE	Used to detect day rollover for quota reset
posts_today	INTEGER	DEFAULT 0	Daily post counter (max 3)
last_post_date	DATE	NULLABLE	Used for daily reset detection
created_at	DATETIME	DEFAULT utcnow()	Brand creation timestamp

owner_id	INTEGER	FK → users.id	Establishes brand ownership
----------	---------	---------------	-----------------------------

Table: posting_plans

Column	Type	Constraints	Description
id	INTEGER	PK, auto-increment	Unique plan identifier
brand_id	INTEGER	FK → brands.id, UNIQUE	One-to-one relationship with brands
active_days	JSON (Array)	NOT NULL	Days of the week to post
time_slots	JSON (Array)	NOT NULL	Times of day to publish (Max 3)
volume	VARCHAR	NOT NULL	Preset: chill, growth, or viral
is_active	BOOLEAN	DEFAULT TRUE	Toggle for scheduling queue

Table: content_items

Column	Type	Constraints	Description
id	INTEGER	PK, auto-increment	Unique content identifier
brand_id	INTEGER	FK → brands.id	Brand ownership association

body	TEXT	NOT NULL	The tweet text (< 280 characters)
status	ENUM	DEFAULT DRAFT	DRAFT, PENDING_APPROVAL, APPROVED, SCHEDULED, PUBLISHED, REJECTED
scheduled_for	DATETIME	NULLABLE	UTC timestamp for publishing
tweet_id	VARCHAR	NULLABLE	Twitter's ID for the published tweet
created_at	DATETIME	DEFAULT utcnow()	Creation timestamp
author_id	INTEGER	FK → users.id	Creator of the content

Content Status Transitions State Machine

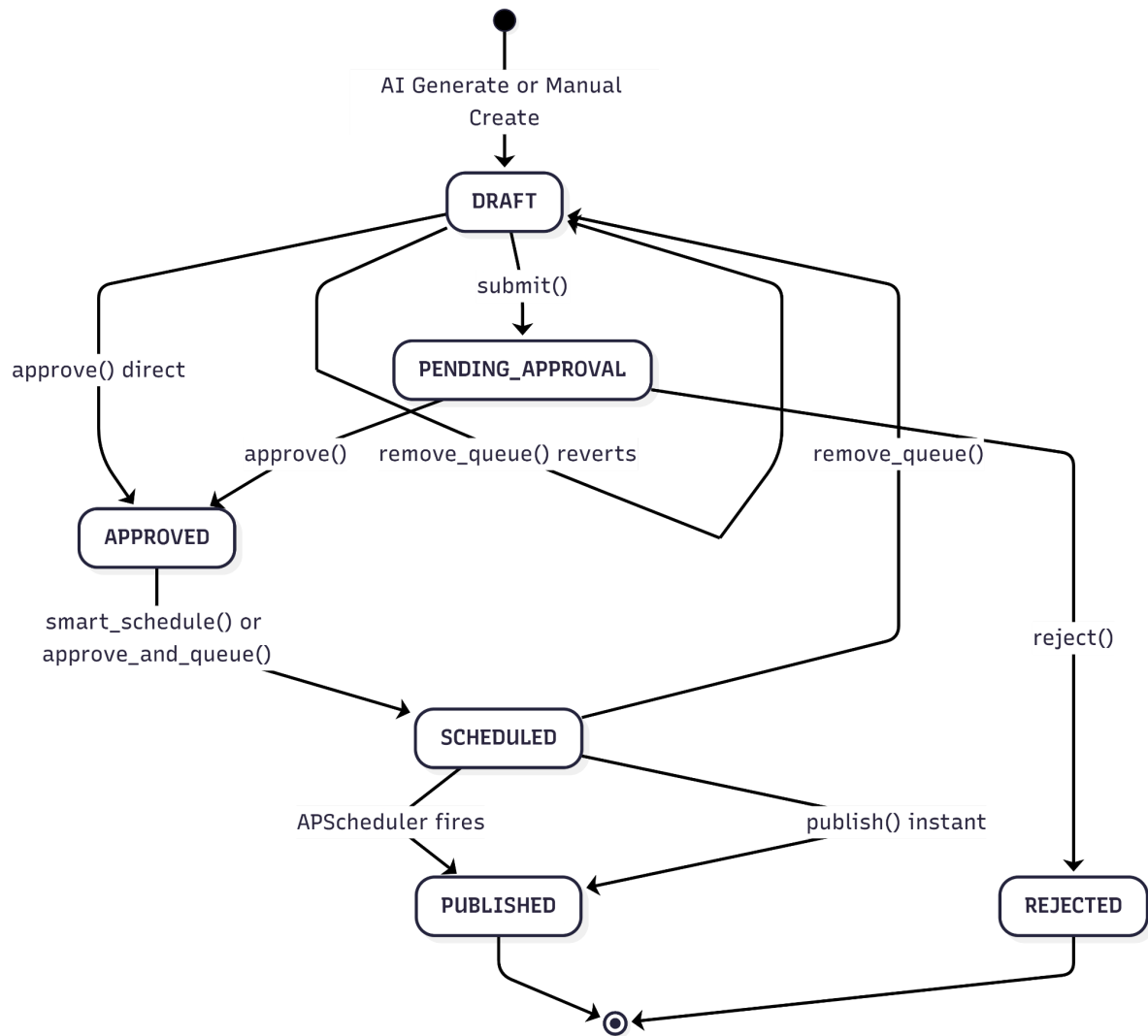


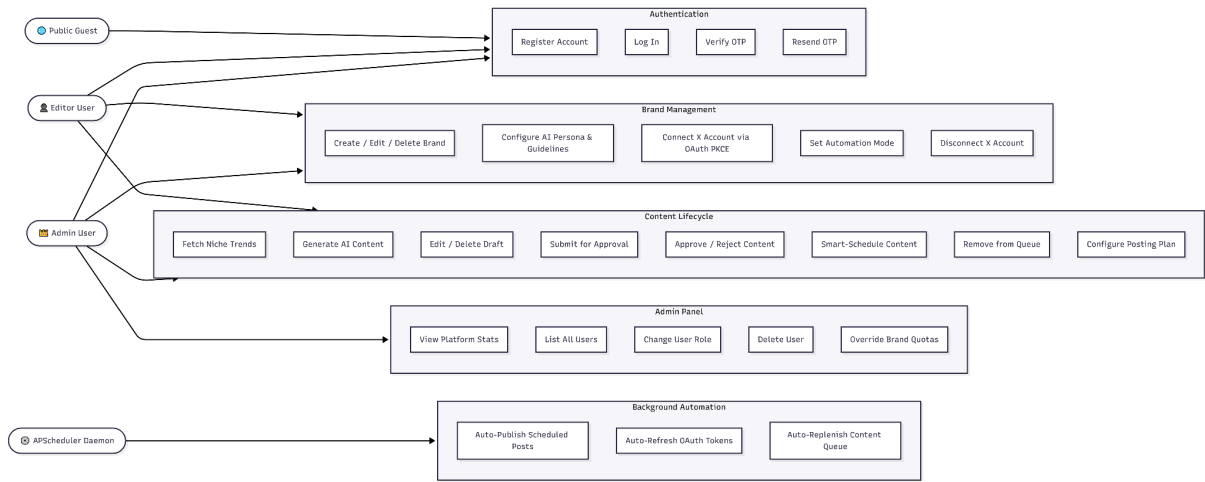
Table: **validation_rules**

Column	Type	Constraints	Description
id	INTEGER	PK, auto-increment	Unique rule identifier
brand_id	INTEGER	FK → brands.id	Brand ownership association
rule_type	VARCHAR	NOT NULL	Identifier (e.g., forbidden_words)

parameters	JSON (Object)	NOT NULL	Rule configuration parameters
------------	---------------	----------	-------------------------------

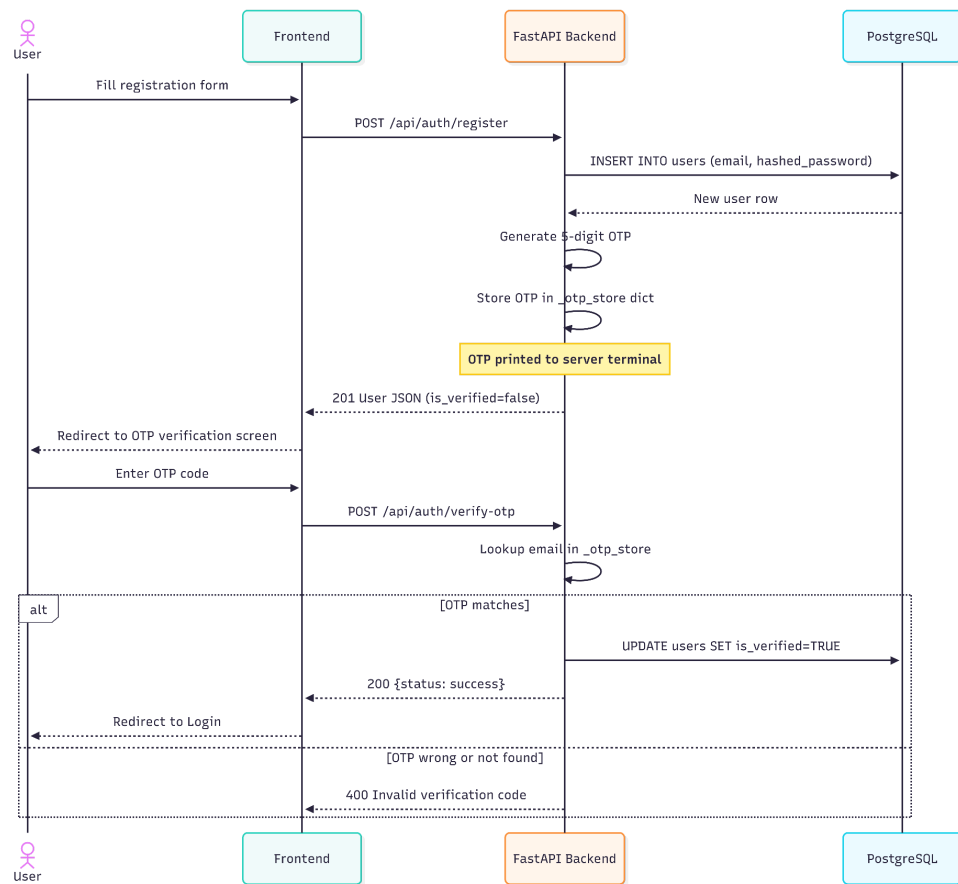
3. System Modeling — User Flows & Logic

3.1 Use Case Diagram

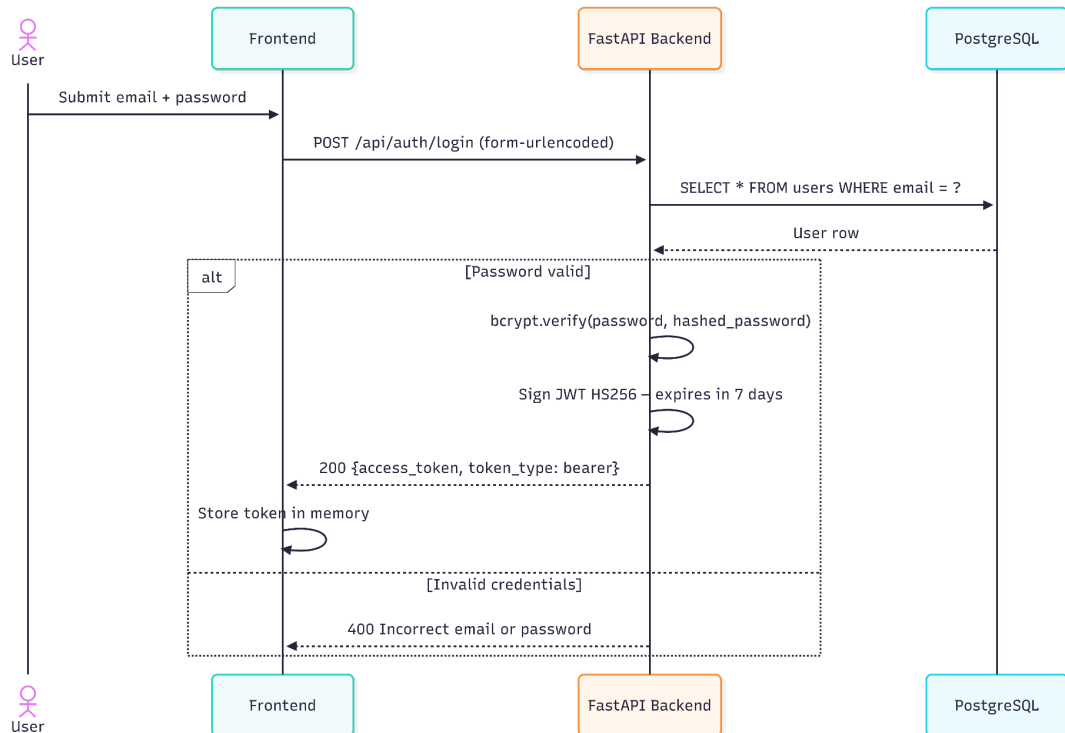


3.2 Sequence Diagrams

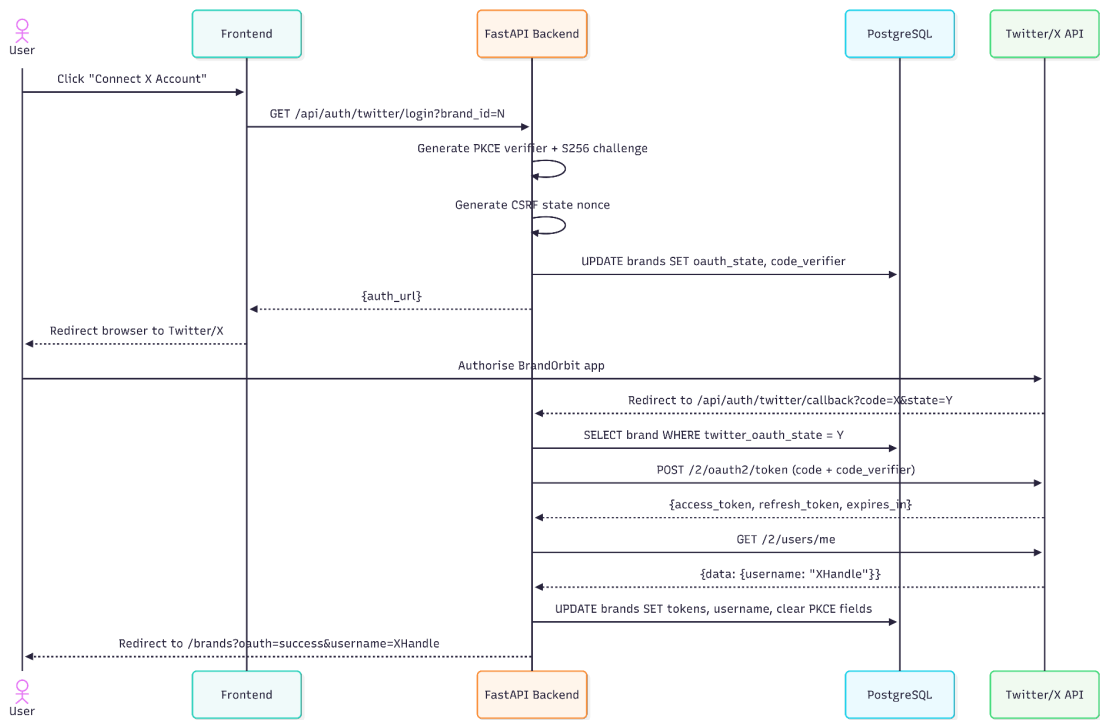
- **User Registration & OTP Verification Sequence**



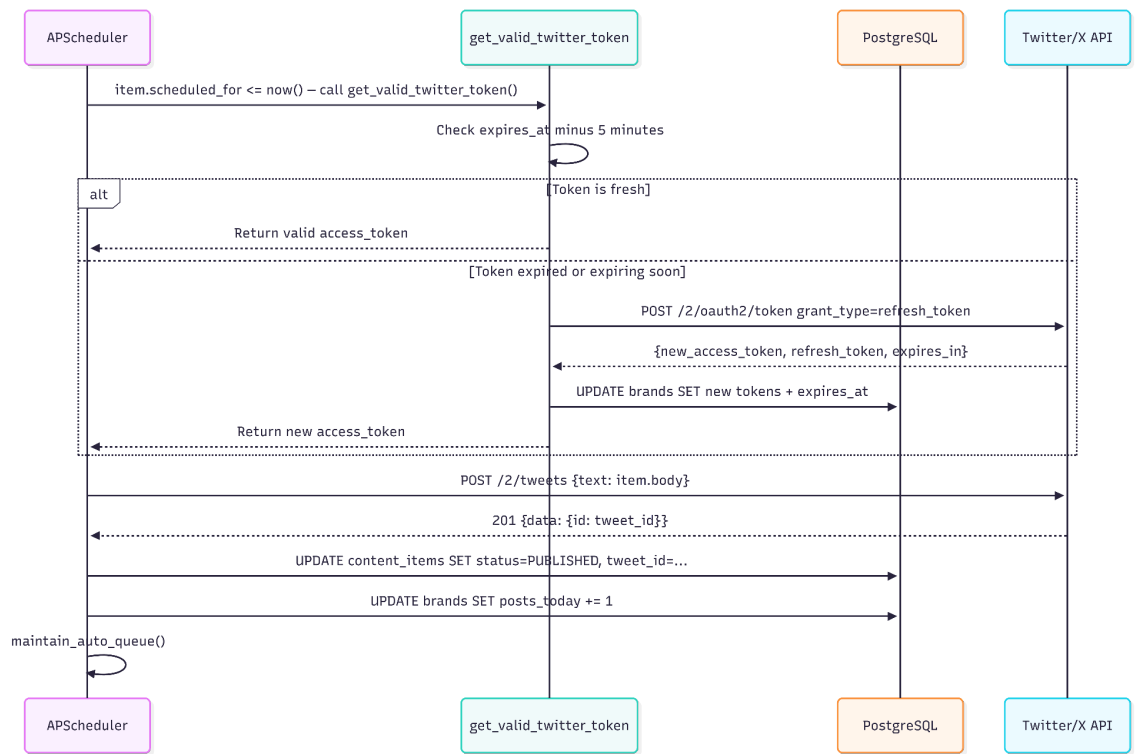
• Login & JWT Issuance Sequence



● **Twitter/X OAuth 2.0 PKCE Flow Sequence**

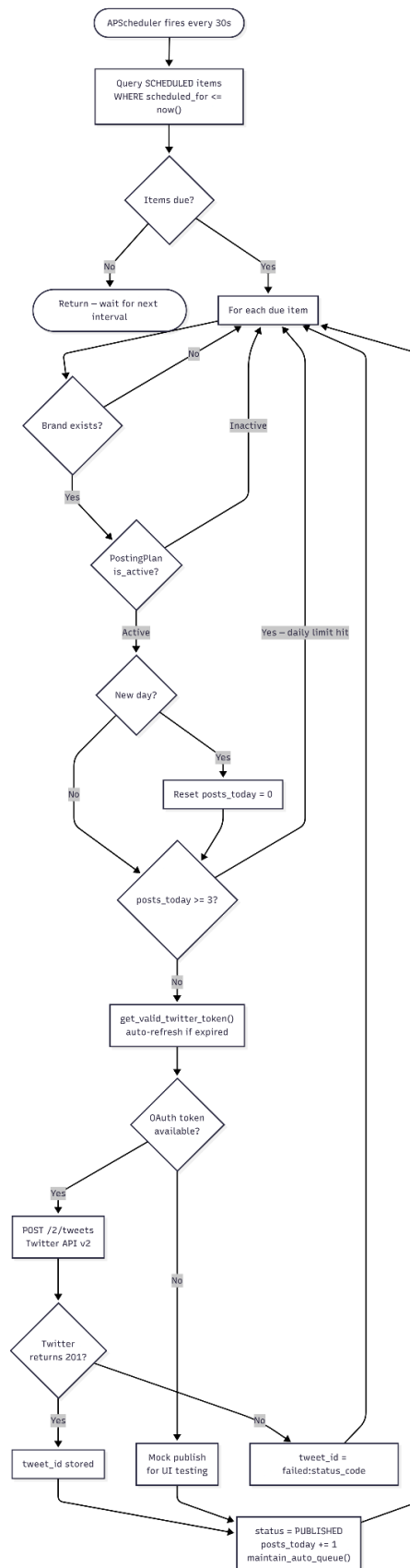


● **AI Content Generation Sequence**

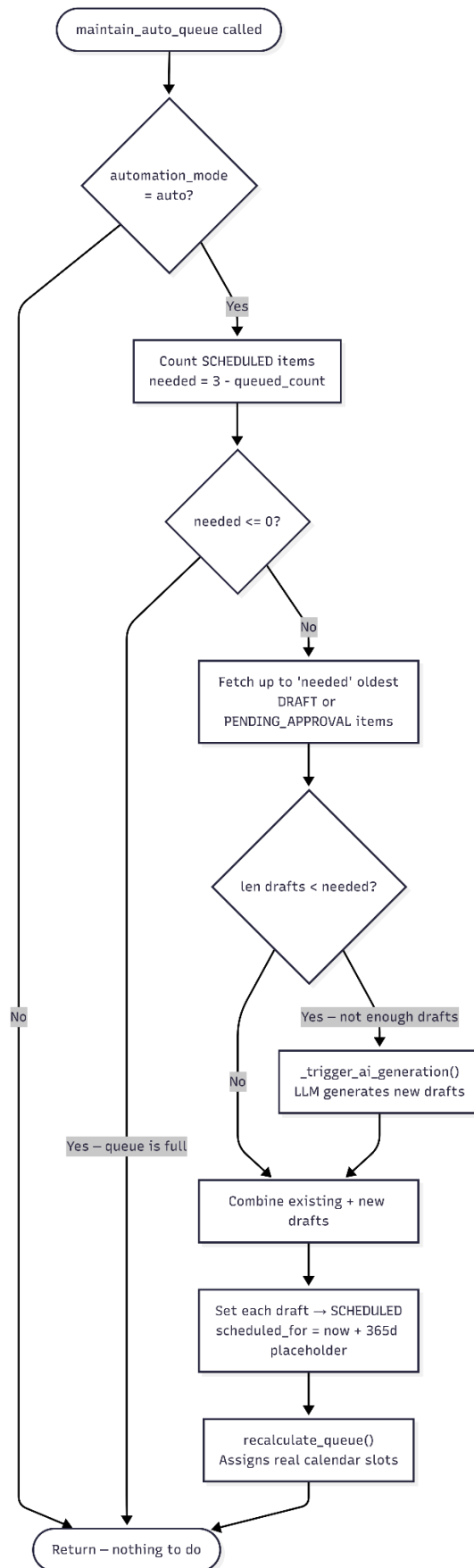


3.3 Activity Diagrams (Background Workers)

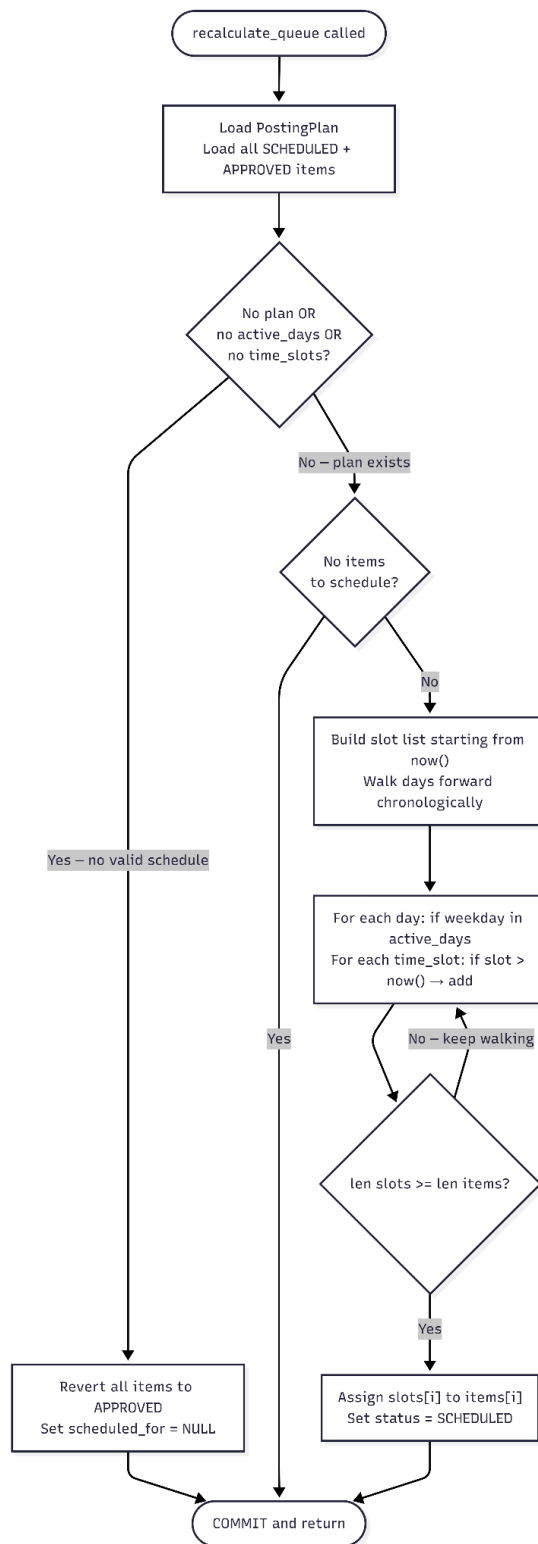
- APScheduler Auto-Pilot Daemon (Every 30s)



- **maintain_auto_queue() : Auto-Pilot Replenishment**



- **recalculate_queue() : Calendar Slot Assignment**



4. API Reference

Base URL: <http://127.0.0.1:8000>

Auth Header Format: [Authorization: Bearer <jwt_token>](#)

4.1 System

- [GET /](#) (Public) - Health check confirming API server is running.
- [POST /api/debug/trigger-scheduler](#) (Public) - Manually fires the publish scheduler immediately.

4.2 Authentication

- [POST /api/auth/register](#) (Public) - Creates a new user account. Expects [email](#) and [password](#). Returns user JSON with [is_verified=false](#).
- [POST /api/auth/login](#) (Public) - Authenticates via [application/x-www-form-urlencoded](#) data ([username](#), [password](#)). Returns [access_token](#) (JWT).
- [POST /api/auth/verify-otp](#) (Public) - Validates 5-digit [otp](#) against [email](#). Sets [is_verified = TRUE](#).
- [POST /api/auth/resend-otp](#) (Public) - Generates and sends a new OTP code for the given [email](#).
- [GET /api/users/me](#) (Authenticated) - Returns the profile of the currently authenticated user.

4.3 Brands

- [POST /api/brands/](#) (Verified) - Creates a brand. Accepts [name](#), [description](#), [niche](#), [quirks](#), [persona_guidelines](#), [automation_mode](#).
- [GET /api/brands/](#) (Verified) - Returns all brands owned by the authenticated user (supports [skip](#), [limit](#) queries).
- [GET /api/brands/{brand_id}](#) (Verified) - Retrieves a single brand by ID.
- [PUT /api/brands/{brand_id}](#) (Verified) - Updates brand fields. Silently ignores OAuth token fields to protect credentials.
- [PUT /api/brands/{brand_id}/mode](#) (Verified) - Updates [automation_mode](#) ([manual](#), [semi-automated](#), [auto](#)).

4.4 Posting Plans

- [GET /api/brands/{brand_id}/plan](#) (Verified) - Fetches the active posting schedule.
- [POST /api/brands/{brand_id}/plan](#) (Verified) - Upserts the posting plan ([active_days](#), [time_slots](#), [volume](#), [is_active](#)). Automatically triggers [recalculate_queue\(\)](#).

4.5 Trends & AI Generation

- [GET /api/brands/{brand_id}/trends](#) (Verified) - Returns 3 AI-refined trending content angles utilizing pytrends + Groq/Gemini. 4-hour cache.
- [POST /api/brands/{brand_id}/generate](#) (Verified) - Triggers AI generation (max 4/day). Accepts optional [trend](#). Returns array of [DRAFT](#) ContentItems.

4.6 Content Lifecycle

- [POST /api/brands/{brand_id}/content](#) (Verified) - Manually creates a [DRAFT](#) content item.

- **GET /api/brands/{brand_id}/content** (Verified) - Returns all content items for a brand ordered by `created_at` DESC.
- **PUT /api/content/{content_id}** (Verified) - Edits the body of a **DRAFT**.
- **DELETE /api/content/{content_id}** (Verified) - Permanently deletes a content item.
- **POST /api/content/{content_id}/submit** (Verified) - Transitions a **DRAFT** to **PENDING_APPROVAL**.
- **POST /api/content/{content_id}/approve** (Verified) - Transitions **DRAFT/PENDING_APPROVAL** to **APPROVED**.
- **POST /api/content/{content_id}/reject** (Verified) - Transitions **PENDING_APPROVAL** to **REJECTED**.
- **POST /api/content/{content_id}/schedule** (Verified) - Manually assigns a `scheduled_for` datetime to an **APPROVED** item.
- **POST /api/content/{content_id}/smart_schedule** (Verified) - Automatically slots an **APPROVED** item into the calendar based on the brand's PostingPlan.
- **POST /api/content/{content_id}/approve_and_queue** (Verified) - Atomic operation: approves and immediately smart-schedules a draft.
- **POST /api/content/{content_id}/remove_queue** (Verified) - Pulls a **SCHEDULED** item back to **DRAFT**.
- **POST /api/content/{content_id}/publish** (Verified) - Instantly publishes an item to Twitter/X, bypassing the scheduler.

4.7 Twitter/X OAuth 2.0

- **GET /api/auth/twitter/login** (Public) - Initiates PKCE flow. Requires `brand_id` query. Returns Twitter authorization URL.
- **GET /api/auth/twitter/callback** (Public) - Twitter callback endpoint. Exchanges code for tokens and redirects browser back to React frontend.
- **POST /api/brands/{brand_id}/disconnect** (Public) - Nullifies the brand's stored Twitter OAuth credentials.

4.8 Validation Rules

- **POST /api/brands/{brand_id}/rules** (Public) - Attaches a validation rule (`rule_type`, parameters).
- **GET /api/brands/{brand_id}/rules** (Public) - Lists all rules for a brand.

4.9 Admin Panel (Role = ADMIN)

- **GET /api/admin/stats** (Admin) - Platform snapshot (user/brand counts, content pipeline, timelines).
- **GET /api/admin/users** (Admin) - Lists users and their full brand portfolios.
- **PUT /api/admin/users/{user_id}/role** (Admin) - Promotes/demotes user roles.
- **DELETE /api/admin/users/{user_id}** (Admin) - Permanently deletes a user and all cascade data.
- **PUT /api/admin/brands/{brand_id}/quota** (Admin) - Manually overrides the `generations_today` and `posts_today` limits.